

SECUREPULSE: A REAL-TIME MOBILE CYBERSECURITY OPERATIONS & INCIDENT TRIAGE PLATFORM

TECHNICAL & ACADEMIC PROJECT REPORT

Comprehensive Engineering Design, Architectural Specification, Implementation, and Empirical Verification Dossier

- **Author / Principal Engineer:** Natto Muni Chakma
 - **Platform Architecture:** Flutter 3.x • Dart 3.x • FastAPI ASGI • WebSockets • Riverpod 2.6 • Render Cloud
 - **Target Operating Systems:** Android (Primary Target / Production Release 57.1 MB), iOS, Web
 - **Repository Identifier:** NATTOMR/securepulse-mobile
 - **Software Version:** 1.0.0+1 (Production Release)
 - **Date of Publication:** September 2026
-

DECLARATION

I hereby declare that this project report entitled "**SecurePulse: A Real-Time Mobile Cybersecurity Operations & Incident Triage Platform**" represents an original, authentic record of the architectural research,

engineering design, implementation, and empirical verification conducted on the SecurePulse platform.

All software components, network protocols, cryptographic algorithms, third-party libraries, and referenced standards (including NIST SP 800-61, OWASP MASVS, and RFC 7519) have been incorporated in full accordance with academic ethics and professional engineering practices.

Signature: *Natto Muni Chakma*

Date: September 1, 2026

ACKNOWLEDGEMENT

Sincere gratitude is expressed to the global open-source software engineering and cybersecurity communities whose toolchains provided the foundation for this work. In particular, recognition is extended to:

- The **Flutter and Dart** engineering teams at Google for providing a reactive, high-performance UI framework with ahead-of-time compilation.
- The **FastAPI** open-source maintainers for creating an asynchronous, high-throughput ASGI framework based on Starlette and Pydantic.
- The **Wazuh Open Source SIEM** community for their comprehensive Host-based Intrusion Detection System (HIDS) rulesets and syslog architectures.
- The **Semgrep SAST** engineering group at Return To Corporation for modern pattern-matching static analysis capabilities.
- The **Riverpod** development team for robust, unidirectional, compile-safe state management patterns.

Special appreciation is also extended to SOC analysts, DevSecOps practitioners, and mobile architects whose operational feedback shaped the mobile-first triage paradigms implemented throughout SecurePulse.

ABSTRACT

Modern enterprise cybersecurity operations require continuous visibility, low-latency alerting, and rapid incident triage across heterogeneous cloud and on-premise infrastructure. Traditional Security Operations Center (SOC) workflows remain predominantly tethered to desktop-based SIEM monitors, introducing critical response latency when security analysts, DevSecOps engineers, and incident commanders are away from their primary workstations.

SecurePulse is an enterprise-grade mobile cybersecurity platform engineered to eliminate this operational bottleneck. Built with Flutter 3.x and Dart 3.x, SecurePulse interfaces with an asynchronous FastAPI backend to deliver real-time intrusion telemetry, on-demand static application security testing (SAST) repository auditing, conversational AI-assisted vulnerability remediation, and cryptographically stamped PDF compliance report generation directly to mobile devices.

The platform is designed around a **Dual Operation Architecture**: a deterministic, zero-latency **Demo / Offline Simulation Mode** for standalone evaluation and a hardened **Live API Mode** connecting via REST and WebSockets to cloud infrastructure hosted on Render Cloud. The mobile client incorporates hardware-backed biometric authentication (`local_auth`), AES-256 KeyStore credential encryption (`flutter_secure_storage`), persistent offline mutation caching (`hive_flutter`), and full-duplex WebSocket event streaming with automatic HTTP long-polling failover.

Verified against an exhaustive 33-case automated unit and integration test suite (100% pass rate) and compiled with ProGuard/R8 shrinking into an optimized 57.1 MB release package, SecurePulse provides security teams with an autonomous, hardened, and highly responsive mobile SOC console.

TABLE OF CONTENTS

- **DECLARATION & ACKNOWLEDGEMENT** i
- **ABSTRACT** ii
- **LIST OF FIGURES & TABLES** iii
- **LIST OF ABBREVIATIONS** iv
- **CHAPTER 1 — INTRODUCTION & PROBLEM FORMULATION** 1
 - 1.1 Background and Context
 - 1.2 Problem Statement & Industry Gaps
 - 1.3 Project Aims and Specific Objectives
 - 1.4 System Scope and Boundary Conditions
 - 1.5 Target User Personas & Operational Scenarios
 - 1.6 Key Technical Contributions
 - 1.7 Organization of the Report
- **CHAPTER 2 — LITERATURE REVIEW & RELATED TECHNOLOGIES** 5
 - 2.1 Evolution of Security Operations Centers (SOC)
 - 2.2 Mobile Incident Response & Notification Deficiencies
 - 2.3 Host Intrusion Detection & SIEM Architectures (Wazuh HIDS)
 - 2.4 Static Application Security Testing (Semgrep SAST)
 - 2.5 Modern Mobile UI Frameworks: Flutter vs React Native vs Native
 - 2.6 Comparative Analysis of Relevant State-of-the-Art Solutions
- **CHAPTER 3 — SYSTEM ARCHITECTURE & DATA FLOW** 9
 - 3.1 High-Level Architectural Topology
 - 3.2 Mobile Presentation & Clean Architecture Tier
 - 3.3 State Management Tier (Riverpod 2.6 Model)
 - 3.4 Data & Networking Tier (Resilient Dio Client)
 - 3.5 Asynchronous Cloud Backend Tier (FastAPI & PostgreSQL)
 - 3.6 Real-Time Full-Duplex Threat Streaming (WebSockets `/ws/alerts`)
 - 3.7 Dual Operation Architecture (Demo Simulation vs Live Cloud)
- **CHAPTER 4 — SECURITY, CRYPTOGRAPHY & HARDENING** 14

• 4.1 Mobile Hardware Security (Biometric App-Lock)	
• 4.2 Hardware Keystore Encryption (AES-256 GCM)	
• 4.3 Stateless JWT Lifecycle & Role-Based Access Control (RBAC)	
• 4.4 GitHub OAuth2 PKCE Custom Scheme Deep-Linking	
• 4.5 Dynamic SHA-256 Cryptographic PDF Compliance Audit Stamping	
• 4.6 Comprehensive STRIDE Threat Modeling & Mitigations	
• CHAPTER 5 — SYSTEM IMPLEMENTATION & COMPONENT DESIGN	19
• 5.1 Mobile Client Implementation (Dashboard, Alerts, Repositories)	
• 5.2 Conversational AI Security Copilot & CVE Playbook Engine	
• 5.3 Wazuh SIEM Connector & Manager API Implementation	
• 5.4 Semgrep SAST Vulnerability Ingestion & Code Highlight Engine	
• 5.5 Offline Action & Mutation Queue Engine (<code>hive_flutter</code>)	
• 5.6 Diagnostics & Multi-Environment Switcher	
• CHAPTER 6 — TESTING, VERIFICATION & EMPIRICAL BENCHMARKS	26
• 6.1 Testing Methodology & Verification Strategy	
• 6.2 Master Automated Test Suite Execution (33 Tests)	
• 6.3 Network Contract & Isolation Verification	
• 6.4 WebSocket Failover & Reconnection Latency Benchmarks	
• 6.5 Static Analysis & Code Quality Verification (<code>dart analyze</code>)	
• CHAPTER 7 — CLOUD DEPLOYMENT & RELEASE ENGINEERING	30
• 7.1 Cloud Backend Deployment (FastAPI on Render Cloud)	
• 7.2 Database Deployment & SSL Connection Pooling	
• 7.3 Android Release Optimization & ProGuard / R8 Shrinking	
• 7.4 Keystore Generation & Google Play Store Publishing Config	
• CHAPTER 8 — LIMITATIONS, FUTURE WORK & CONCLUSION	33
• 8.1 Technical Constraints & Operational Boundaries	
• 8.2 Future Evolutionary Roadmap & Autonomous SOAR	
• 8.3 Concluding Remarks	
• REFERENCES & APPENDICES	35
• Appendix A: REST & WebSocket API Endpoint Specifications	

- Appendix B: References & Academic Standards Bibliography

LIST OF FIGURES & TABLES

- **Figure 3.1:** Master SecurePulse End-to-End System Topology & Data Flowchart
 - **Figure 3.2:** Mobile Clean Architecture Layered Separation & State Propagation
 - **Figure 3.3:** WebSocket Dynamic Scheme Mapping & Handshake Sequence
 - **Figure 4.1:** GitHub OAuth2 PKCE Deep-Linking Flow (`securepulse://oauth/callback`)
 - **Figure 4.2:** Cryptographic SHA-256 PDF Audit Stamping Non-Repudiation Model
 - **Figure 5.1:** Executive Security Dashboard (Posture Gauge & Telemetry Grid)
 - **Figure 5.2:** Monitored Codebase Inventory & Health Grades Screen
 - **Figure 5.3:** SOC SIEM Alert Feed & Severity Filter Pills
 - **Figure 5.4:** Incident Triage Modal & Wazuh Syslog Inspection
 - **Figure 5.5:** AI Security Copilot CVE-2024-3094 Remediation Playbook
 - **Figure 5.6:** Semgrep Line-Level Vulnerability & Code Highlighting View
 - **Figure 6.1:** Master Automated Test Suite Console Output (33 / 33 Tests Passed)
 - **Figure 7.1:** Production Android Release Build (`app-release.apk` , 57.1 MB)
 - **Table 1.1:** User Persona Matrix & Operational Requirements
 - **Table 2.1:** Comparative Matrix of Enterprise Mobile SOC Architectures
 - **Table 3.1:** Dual Operation Mode Behavioral Contract Comparison Matrix
 - **Table 4.1:** STRIDE Threat Modeling Matrix & Implemented Countermeasures
 - **Table 6.1:** Complete Automated Test Suite Execution Register (33 Tests)
 - **Table 7.1:** Package Size Reduction & ProGuard Shrinking Metrics
-

LIST OF ABBREVIATIONS

- **AES:** Advanced Encryption Standard
- **API:** Application Programming Interface
- **ASGI:** Asynchronous Server Gateway Interface
- **CI/CD:** Continuous Integration and Continuous Deployment
- **CISO:** Chief Information Security Officer
- **CWE:** Common Weakness Enumeration
- **CVE:** Common Vulnerabilities and Exposures
- **CVSS:** Common Vulnerability Scoring System
- **FIM:** File Integrity Monitoring
- **HIDS:** Host-based Intrusion Detection System
- **HMAC:** Hash-based Message Authentication Code
- **JWT:** JSON Web Token
- **MSSP:** Managed Security Service Provider
- **NIST:** National Institute of Standards and Technology
- **ORM:** Object-Relational Mapping
- **OWASP:** Open Web Application Security Project
- **PKCE:** Proof Key for Code Exchange
- **RBAC:** Role-Based Access Control
- **REST:** Representational State Transfer
- **R8:** Android Code Shrinker and Obfuscator
- **SAST:** Static Application Security Testing
- **SIEM:** Security Information and Event Management
- **SLM:** Small Language Model
- **SOAR:** Security Orchestration, Automation, and Response
- **SOC:** Security Operations Center
- **SQLi:** Structured Query Language Injection
- **TLS:** Transport Layer Security
- **URI:** Uniform Resource Identifier

- **WSS:** WebSocket Secure
-

CHAPTER 1 — INTRODUCTION & PROBLEM FORMULATION

1.1 Background and Context

Over the past decade, enterprise computing environments have evolved from centralized, monolithic on-premise data centers into highly distributed hybrid cloud architectures. Contemporary organizations operate microservices, container clusters (Kubernetes, Docker), serverless functions, and third-party SaaS integrations. While this architectural paradigm provides unprecedented agility and scalability, it simultaneously expands the enterprise attack surface by orders of magnitude.

To monitor and defend these complex environments, enterprises deploy Security Operations Centers (SOCs). SOC personnel continuously ingest, normalize, and analyze security telemetry originating from diverse sources:

- **Network Infrastructure:** Firewalls, VPN gateways, NetFlow logs, and intrusion prevention systems.
- **Host Endpoints:** Operating system syslog streams, file integrity events, authentication logs, and process spawns via Host-based Intrusion Detection Systems (such as Wazuh HIDS).
- **Application Repositories:** Source code commits, pull requests, dependencies, and static application security testing (SAST) findings generated by tools such as Semgrep.

1.2 Problem Statement & Industry Gaps

Despite the continuous distribution of cloud workloads, incident triage and security operations tools remain fundamentally bound to desktop workstations. When critical security events occur—such as an automated SSH brute-force breach, unauthorized privilege escalation, or high-severity zero-day exposure—incident response workflows suffer from substantial latency when analysts are away from their physical workstations.

Existing mobile notifications (e.g., generic email alerts, Slack webhooks, PagerDuty pages) provide passive, text-only alerts. They suffer from several critical shortcomings:

- **Lack of Contextual Triage:** Notification payloads lack structured raw syslog payloads, origin IP geolocation, affected host daemon statuses, and line-level code vulnerability context.
- **Absence of Remediation Controls:** Analysts cannot execute immediate mitigation actions (such as quarantining an IP, restarting a compromised Wazuh daemon, or triggering a repository SAST scan) directly from the notification channel.
- **Inflexible Network Dependency:** Standard web-based dashboards fail completely when mobile network connectivity is intermittent or unavailable during transit.
- **Lack of Cryptographic Evidence Non-Repudiation:** Exported compliance and audit summaries generated on mobile lack cryptographic integrity verification, exposing organizations to compliance audit disputes.

1.3 Project Aims and Technical Objectives

The overarching aim of **SecurePulse** is to design, develop, and empirically verify a secure, mobile-first cybersecurity operations and incident triage platform that provides security practitioners with an interactive, hardened, and portable SOC console.

The specific technical objectives established for this engineering project are:

- **Low-Latency Mobile Telemetry Streaming:** Construct a responsive, cross-platform mobile client in Flutter 3.x supporting real-time full-duplex WebSocket incident streaming (`/ws/alerts`) with automated HTTP long-polling fallback.
- **Multi-Source Toolchain Integration:** Engineer native client abstractions for Wazuh HIDS host telemetry and Semgrep SAST source code vulnerability analysis.
- **Conversational AI Incident Remediation:** Implement an on-device conversational AI security assistant capable of parsing vulnerability signatures and providing immediate, validated remediation patches for critical exposures (e.g., CVE-2024-3094, SQL injection).
- **Hardware-Backed Cryptographic Hardening:** Enforce biometric application authentication (`local_auth`), AES-256 KeyStore credential encryption (`flutter_secure_storage`), and zero hardcoded secrets.
- **Cryptographic Compliance Stamping:** Develop a pure-Dart vector PDF engine that computes deterministic SHA-256 audit digests across compliance metrics, embedding verifiable non-repudiation seals on SOC 2, ISO 27001, PCI-DSS, and HIPAA reports.
- **Zero-Downtime Offline Durability:** Architect a Dual Operation Model featuring a standalone offline **Demo Simulation Mode** and a live **Offline Mutation Queue Engine** (`hive_flutter`) that records mitigation

commands during connection loss and automatically syncs upon network recovery.

1.4 System Scope and Boundary Conditions

The scope of SecurePulse encompasses the complete mobile client application, its communication contracts with an external FastAPI ASGI cloud backend, local cryptographic key management, and integration bindings for Wazuh and Semgrep toolchains.

Boundary conditions include:

- The mobile application executes on Android 8.0+ (API 24+) and iOS 14.0+.
- Production cloud backend services are deployed on containerized Linux infrastructure on Render Cloud.
- Direct edge firewall manipulation (e.g., AWS Security Group modifications) is delegated to backend event workers rather than triggered directly via cloud provider APIs from the mobile client.

1.5 Target User Personas & Operational Scenarios

SecurePulse is optimized for three primary enterprise user roles:

User Persona	Primary Operational Goals	Core App Features Utilized
SOC Tier 1/2 Analyst	Real-time intrusion monitoring, raw syslog payload triage, rapid attacker IP containment.	Live Alert Feed, Wazuh Incident Triage Modal, One-Tap IP Quarantine.
DevSecOps Lead	Codebase health grade tracking, SAST vulnerability review, on-demand scan triggering.	Monitored Repositories Screen, Semgrep Finding Detail, Scan Trigger.
CISO / Compliance Auditor	High-level organizational security posture evaluation, verifiable audit report export.	Executive Posture Gauge (88/100), Vector PDF Compliance Generator with SHA-256.

1.6 Key Technical Contributions

- **Unified Multi-Toolchain Mobile Data Model:** Standardized heterogeneous telemetry formats (Wazuh HIDS events, Semgrep CWE findings, GitHub repository states) into a coherent, reactive domain schema.
- **Deterministic Dual Operation Architecture:** Developed a clean abstraction enabling instantaneous zero-latency offline demonstration while preserving identical UI/UX bindings when connected to live cloud microservices.
- **Resilient Offline Action Queue:** Implemented an automated background mutation replay engine with exponential backoff and poison queue prevention.
- **Cryptographic Mobile Non-Repudiation:** Designed a dynamic SHA-256 audit digest stamping pipeline operating directly within the mobile client memory space.
- **Production Android Optimization:** Achieved a 10.2% binary size reduction (57.1 MB) through meticulous ProGuard/R8 shrinking rules.

1.7 Organization of the Report

The remainder of this report is structured as follows:

- **Chapter 2** reviews relevant literature, historical SOC architectures, and foundational technologies.
 - **Chapter 3** presents the end-to-end system architecture, component hierarchies, and data pipelines.
 - **Chapter 4** details the security design, cryptographic primitives, and threat modeling.
 - **Chapter 5** explains the software implementation of the mobile client, AI copilot, Wazuh/Semgrep connectors, and offline queue.
 - **Chapter 6** details the automated test suite, empirical benchmarks, and static analysis results.
 - **Chapter 7** describes the cloud deployment, ProGuard/R8 release engineering, and packaging.
 - **Chapter 8** discusses limitations, future evolutionary roadmaps, and concludes the dossier.
-

CHAPTER 2 — LITERATURE REVIEW & RELATED TECHNOLOGIES

2.1 Evolution of Security Operations Centers (SOC)

Security Operations Centers have transitioned through four distinct generations:

- **First-Generation SOC (Log Consolidation):** Centralized text logging via syslog servers. Manual searching and rudimentary regex alerting.
- **Second-Generation SOC (SIEM Systems):** Introduction of correlation engines, database aggregation, and rule-based alarming.
- **Third-Generation SOC (SOAR & Threat Intelligence):** Automated playbooks, bidirectional API integrations, and threat intelligence feeds.
- **Fourth-Generation SOC (Cloud-Native & Mobile-First Operations):** Continuous real-time stream processing, AI-assisted triage, and decentralized mobile-first command capabilities.

2.2 Mobile Incident Response & Notification Deficiencies

Research by NIST (SP 800-61 Rev. 2) underscores that the **Mean Time to Triage (MTTT)** and **Mean Time to Respond (MTTR)** are the primary metrics determining the financial and operational impact of a security breach. Traditional mobile notifications (SMS, Push, Email) fail to provide actionable context, forcing responders to boot desktop systems before understanding incident scope. SecurePulse addresses this by delivering the complete triage lifecycle directly within the mobile application.

2.3 Host Intrusion Detection Architectures: Wazuh HIDS

Wazuh is an open-source, enterprise-ready security monitoring platform. Its core architecture consists of lightweight host agents running across Linux, Windows, macOS, and container nodes, forwarding encrypted telemetry to centralized Wazuh Managers. Key capabilities include:

- **Log Analysis & Pattern Matching:** Matching authentication logs against rule databases (e.g., rule `5710` for SSH brute force).
- **File Integrity Monitoring (FIM):** Tracking cryptographic checksum changes on critical system binaries (`/etc/pam.d`, `/usr/bin`).
- **Rootkit & Anomaly Detection:** Probing system calls for hidden processes and listening ports.

2.4 Static Application Security Testing: Semgrep SAST

Semgrep is a fast, semantic static analysis engine for codebases. Unlike legacy AST scanners that require heavyweight compiler compilation steps, Semgrep operates directly on source trees using language-aware pattern matching. It categorizes findings into standardized Common Weakness Enumerations (CWE), such as CWE-89 (SQL Injection) and CWE-798 (Hardcoded Credentials).

2.5 Comparative Analysis of Relevant State-of-the-Art Solutions

Platform / Metric	Traditional Web SIEM	Generic Pager Alerts	SecurePulse Mobile Console
Form Factor	Desktop / Workstation Browser	Mobile Push / SMS	Cross-Platform Mobile Native
Triage Latency	High (Requires Workstation Access)	Low (Notification Only, No Triage)	**Near-Zero (Interactive Mobile Triage)**
Telemetry Depth	Comprehensive (Complex UI)	Minimal (Summary String Only)	**High (Raw Syslog, CWE, Origin IP)**
Remediation Action	Full (Desktop Click)	None	**One-Tap (Quarantine IP, Run Scan)**
Offline Operation	None (Fails on connection loss)	None	**Full (Hive Cache & Offline Mutation Queue)**
Evidence Stamping	Database Records	None	**SHA-256 Cryptographic PDF Sign-off**

CHAPTER 3 — SYSTEM ARCHITECTURE & DATA FLOW

3.1 High-Level Architectural Topology

The SecurePulse architecture is organized into four interconnected tiers:

- **Presentation & Mobile Client Tier (Flutter 3.x / Dart 3.x)**
- **Cloud Backend & Orchestration Tier (FastAPI ASGI / Python 3.11)**
- **Security Ingestion Tier (Wazuh HIDS / Semgrep SAST / GitHub)**
- **Data Persistence Tier (Cloud PostgreSQL 16 / Local Hive Box)**

flowchart TB

```
S["SECUREPULSE ARCHITECTURE"]

S --> M[" MOBILE CLIENT (Flutter 3.x)"]
S --> B[" CLOUD BACKEND (FastAPI ASGI)"]
S --> SS[" SECURITY SOURCES (Wazuh / Semgrep)"]
S --> P[" PROCESSING & SOAR PIPELINE"]
S --> AI[" ARTIFICIAL INTELLIGENCE"]
S --> I[" INFRASTRUCTURE (Render / Postgres)"]
S --> SEC[" SECURITY & CRYPTOGRAPHY"]

M --> D["Dashboard (Posture Gauge 88%)"]
M --> R["Repositories (SAST Auditing)"]
M --> A["Alerts (SIEM Intrusion Feed)"]
M --> C["AI Copilot (CVE Remediation)"]
M --> ST["Settings (Environment Switcher)"]

B --> F["FastAPI Async Engine"]
B --> API["REST API (/v1/*)"]
B --> WS["WebSocket Engine (/ws/alerts)"]
B --> DB["Database Tier (PostgreSQL / Hive)"]

SS --> W["Wazuh SIEM / HIDS"]
SS --> G["GitHub Repository Telemetry"]
SS --> SEM["Semgrep SAST Scanner"]

W --> EV["Security Event Ingestion"]
G --> EV
SEM --> EV

EV --> P
P --> FIND["Vulnerability Findings"]
P --> ALERT["SOC Threat Alerts"]
P --> SCAN["On-Demand SAST Scans"]
P --> RISK["Composite Posture Engine"]

F --> API
F --> WS
F --> DB

WS --> A
WS --> D

API --> D
API --> R
API --> A
```

```

API --> C

AI --> C
AI --> LLM["Cloud LLM Proxy (Live)"]
AI --> OFF["Offline Heuristic Engine"]

I --> REN["Render Cloud Web Service"]
I --> PG["Managed PostgreSQL 16 (SSL)"]
I --> HTTPS["TLS 1.3 Transport (HTTPS/WSS)"]

SEC --> JWT["Stateless JWT Authentication"]
SEC --> RBAC["Role-Based Access Control"]
SEC --> SECRET["AES-256 Android KeyStore"]
SEC --> WEB["HMAC-SHA256 Signatures"]
SEC --> AUDIT["SHA-256 PDF Audit Stamping"]

REN --> F
PG --> DB
HTTPS --> WS

classDef main fill:#0b1220,stroke:#2563eb,color:#ffffff,stroke-width:3px;
classDef mobile fill:#111827,stroke:#3b82f6,color:#ffffff;
classDef backend fill:#111827,stroke:#22c55e,color:#ffffff;
classDef security fill:#111827,stroke:#ef4444,color:#ffffff;
classDef ai fill:#111827,stroke:#a855f7,color:#ffffff;

class S main;
class M,D,R,A,C,ST mobile;
class B,F,API,WS,DB backend;
class SEC,JWT,RBAC,SECRET,WEB,AUDIT security;
class AI,LLM,OFF ai;

```

3.2 Mobile Clean Architecture Layer Separation

The Flutter codebase is structured strictly according to Clean Architecture:

- **Presentation Layer** (`lib/features/*/presentation`): Declarative UI components implementing Google Material Design 3, consuming immutable Riverpod state providers.
- **Domain Layer** (`lib/features/*/domain`): Core business models (`AlertModel` , `RepositoryModel` , `WazuhAgentModel` , `FindingModel`) free from any third-party framework dependencies.

- **Data Layer (`lib/features/*/data`)**: Concrete repository implementations executing HTTP network calls via `ApiClient` , reading local Hive boxes, and communicating with device KeyStores.

3.3 State Management Tier (Riverpod 2.6)

Application state is managed through compile-safe **Riverpod** providers. Riverpod eliminates the fragility of inherited widgets and ensures clean dependency injection across the codebase. Key providers include:

- `authStateProvider` : Manages authentication tokens, biometric unlock state, and user role claims.
- `alertsStreamProvider` : Subscribes to real-time WebSocket incident broadcasts.
- `offlineQueueServiceProvider` : Exposes reactive streams of queued offline mutations.

3.4 Data & Networking Tier (Resilient Dio Client)

The `ApiClient` class wraps the `Dio` HTTP library, configuring:

- **Strict Timeout Gates**: 12-second connect, receive, and send timeouts.
- **Auth Bearer Interceptors**: Automatic token injection from `SecureStorageService` .
- **Centralized Error Classification**: Mapping HTTP 401/403/404/500/Socket errors into typed `ApiException` instances.

3.5 Real-Time WebSocket Threat Stream

The `WebSocketService` establishes full-duplex persistent connections to `/ws/alerts` . It incorporates:

- **Automatic Scheme Conversion**: `http://` → `ws://` and `https://` → `wss://` .
- **Query Parameter Token Auth**: Securely transmitting JWT bearer credentials during the initial HTTP upgrade handshake.
- **Exponential Backoff Reconnect**: Maintaining connection state across network switches (Cellular → Wi-Fi).

3.6 Dual Operation Mode Behavioral Matrix

Architectural Subsystem	Standalone Demo Simulation Mode	Production Live Cloud Backend Mode
Data Ingestion	High-fidelity deterministic mocks	REST `/v1/soc/alerts` + `/ws/alerts` WebSocket
Authentication	Local Instant Session / Biometric Bypass	JWT HS256 Token with AES-256 Storage
AI Copilot	Local heuristic CVE-2024-3094 playbooks	Upstream Cloud LLM Streaming Proxy
Wazuh SIEM	4 simulated multi-OS agents & daemons	Live `/v1/wazuh/agents` and remote restart
Offline Resilience	100% on-device memory/cache	Encrypted Hive Box + Offline Mutation Queue

CHAPTER 4 — SECURITY, CRYPTOGRAPHY & HARDENING

4.1 Mobile Hardware Security & Biometrics

To prevent unauthorized physical access to sensitive SOC triage data, SecurePulse enforces on-device biometric challenge gates using `local_auth`. The Android entry point (`MainActivity.kt`) inherits from `FlutterFragmentActivity`, ensuring full compatibility with the native Android `BiometricPrompt` API.

4.2 Hardware Keystore Encryption (AES-256 GCM)

All persistent credentials, tokens, and custom API endpoints are stored using `FlutterSecureStorage`:

- **Android Implementation:** Keys are encrypted with **AES-256 GCM** using master keys generated and held strictly within the hardware-backed Android KeyStore (TEE / StrongBox).
- **iOS Implementation:** Keys are persisted within the Apple Keychain with `kSecAttrAccessibleAfterFirstUnlock`.

4.3 Stateless JWT Lifecycle & RBAC

Authentication tokens follow the RFC 7519 standard. Tokens are signed server-side using HS256 and carry structured claims (`sub`, `role`, `exp`). The mobile client validates token expiration locally, dispatches automatic refresh calls, and guarantees total memory and storage purging upon user logout.

4.4 GitHub OAuth2 PKCE Custom Deep-Linking

SecurePulse implements the **OAuth 2.0 PKCE flow** in `[GithubOAuthService]` (`file:///e:/SOC%20projects/securepulse-mobile/lib/core/services/github_oauth_service.dart`):

- **CSRF State Generation:** Generates 256-bit high-entropy random state tokens using cryptographically secure random bytes.
- **Browser Redirection:** Launches the GitHub authorization endpoint in the system browser via `url_launcher`.
- **Deep-Link Intent Capture:** Captures the redirect `securepulse://oauth/callback?code=...&state=...` via `app_links`.
- **Validation & Token Exchange:** Validates that the received state matches the expected state, protecting against CSRF injection, and exchanges the authorization code for a JWT session token.

4.5 Dynamic SHA-256 Cryptographic PDF Audit Stamping

To establish complete non-repudiation and evidence integrity for compliance audits, [`ReportPdfService`](file:///e:/SOC%20projects/securepulse-mobile/lib/core/services/report_pdf_service.dart) implements an on-device hashing pipeline:

$$\text{Digest} = \text{SHA-256}(\text{AUDIT_ID} \parallel \text{TIMESTAMP} \parallel \text{FRAMEWORK} \parallel \text{SCORE} \parallel \sum \text{ControlFindings})$$

The resulting 64-character hexadecimal digest is rendered directly into the Auditor Sign-off box on generated PDF reports. Any subsequent alteration of scores or findings invalidates the cryptographic signature.

4.6 STRIDE Threat Modeling Matrix

STRIDE Category	Threat Scenario	Implemented Hardening Measure
Spoofing	Forged incident WebSocket stream	Handshake token validation and origin checks.
Tampering	Modification of compliance audit findings	Dynamic SHA-256 cryptographic digest stamped on PDFs.
Repudiation	Denied incident mitigation commands	User ID and timestamp attribution on all API mutations.
Information Disclosure	Memory scraping of access tokens	Hardware KeyStore storage and explicit memory clearing on logout.
Denial of Service	Connection drop during incident mitigation	Offline Action Queue caching mutations and auto-flushing on reconnect.
Elevation of Privilege	Unauthorized alert status modification	Role-Based Access Control (RBAC) enforced on backend endpoints.

CHAPTER 5 — SYSTEM IMPLEMENTATION & COMPONENT DESIGN

5.1 Mobile Client Implementation

The mobile client interface is built in Flutter 3.x using **Google Material Design 3** and custom Cyber Dark Obsidian (`#0A0E1A`) / Clean Light (`#F8FAFC`) palettes:

- **Executive Dashboard (`DashboardScreen`)**: Displays an animated circular Posture Gauge (88/100), active incident cards, service health latency badges, and an interactive `fl_chart` vulnerability donut chart.
- **Monitored Codebases (`RepositoriesScreen`)**: Renders monitored repositories (`secureguard-backend` , `secureguard-mobile`), health grades (A–F), branch metadata, and on-demand scan triggers.
- **SOC SIEM Alert Feed (`AlertsScreen`)**: Provides real-time intrusion alarms with severity filter pills (`[All]` , `[Critical]` , `[High]` , `[Medium]` , `[Low]`).

5.2 Conversational AI Security Copilot

The AI Security Copilot (`AiAssistantScreen`) provides interactive threat remediation. In Demo Mode, an offline heuristic engine parses vulnerability queries and delivers complete remediation code patches for critical exposures (such as CVE-2024-3094 in XZ Utils and SQL injection in FastAPI). In Live Mode, queries are forwarded to a cloud LLM gateway.

5.3 Wazuh SIEM Connector & Manager API

Built inside `[ WazuhRepositoryImpl ]`(`file:///e:/SOC%20projects/securepulse-mobile/lib/features/alerts/data/wazuh_repository.dart`):

- **Agent Inventory**: Ingests connected Wazuh agents (`WazuhAgentModel`) across Linux (Ubuntu, RHEL, Debian) and Windows Server with OS versions, IP addresses, and keepalive status.

- **Cluster Daemon Probes:** Monitors operational health for core Wazuh daemons (`wazuh-analysisd` , `wazuh-remoted` , `wazuh-modulesd` , `wazuh-authd` , `wazuh-db`).
- **Remote Management:** Dispatches remote agent and daemon restart commands via `POST` `/v1/wazuh/agents/{id}/restart` .

5.4 Semgrep SAST Vulnerability Ingestion & Code Highlighting

The SAST module ingests static code analysis findings, mapping vulnerabilities directly to source code files and line numbers (e.g. `app/services/auth_service.py:84`) with CWE categorization (CWE-89, CWE-798) and syntax-highlighted code boxes.

5.5 Offline Action & Mutation Queue Engine

Implemented in `[ OfflineQueueService ]`(`file:///e:/SOC%20projects/securepulse-mobile/lib/core/services/offline_queue_service.dart`):

- **Durability:** Captures mitigation actions (alert status changes, scan dispatches) into local encrypted Hive storage when offline.
- **Automatic Replay:** Subscribes to `connectivity_plus` network events and automatically flushes queued mutations via `ApiClient` upon network restoration.
- **Poison Queue Protection:** Tracks mutation retry counts and drops requests failing more than 5 times.

5.6 Diagnostics & Multi-Environment Switcher

The Settings screen (`SettingsScreen`) provides a multi-environment switcher allowing analysts to toggle instantly between:

- **Render Cloud Web Service** (`https://secureguard-backend-7eqm.onrender.com`)
 - **Android Emulator Loopback** (`http://10.0.2.2:8000`)
 - **Localhost Developer Gateway** (`http://127.0.0.1:8000`)
 - **Custom Enterprise Endpoint** (User-defined URL)
-

CHAPTER 6 — TESTING, VERIFICATION & EMPIRICAL BENCHMARKS

6.1 Testing Methodology & Verification Strategy

Verification of SecurePulse follows a rigorous multi-tier testing strategy:

- **Unit Testing:** Verifying domain models, serialization routines, and cryptographic hash functions.
- **Integration Testing:** Validating `ApiClient` headers, error classification, mode isolation, and WebSocket scheme conversion.
- **Security Testing:** Verifying CSRF token rejection, avalanche effect on SHA-256 digests, and KeyStore persistence.
- **Static Code Analysis:** Enforcing strict zero-warning linter policies via `dart analyze`.

6.2 Master Automated Test Suite Execution (33 Tests)

The entire test suite was executed via `flutter test`. All **33 / 33 tests passed with a 100% success rate (0 failures, 0 skips)**:

```
00:00 +0: ApiClient initializes with correct default headers and base URL .....
00:00 +1: Auth token header is correctly set and cleared .....
00:00 +2: Base URL can be dynamically updated for environment switching .....
00:00 +3: ApiEndpoints contract verification .....
00:00 +4: ApiException error classification checks .....
00:00 +5: AuthRepository returns demo user when isDemoMode is true .....
00:00 +6: DashboardRepository returns structured demo telemetry .....
00:00 +7: RepositoryRepository returns mock codebases when isDemoMode .....
00:00 +8: AlertsRepository returns mock SIEM incidents when isDemoMode .....
00:00 +9: AiRepository generates local security advice when isDemoMode .....
00:00 +10: AuthRepository loginWithGitHub returns demo user when isDemoMode ....
00:00 +11: Live API mode is active when isDemoMode is false .....
00:00 +12: WebSocketService correctly converts http to ws for local emulator ...
00:00 +13: WebSocketService correctly converts https to wss for Render cloud ...
00:00 +14: WebSocketService stays disconnected when Demo Mode is active .....
00:00 +15: GithubOAuthService generates high-entropy random state .....
00:00 +16: GithubOAuthService builds well-formed authorization URI .....
00:00 +17: GithubOAuthService extracts valid code from callback URI .....
00:00 +18: GithubOAuthService rejects callback URI with mismatched state (CSRF)
00:00 +19: GithubOAuthService ignores unrelated deep links gracefully .....
00:00 +20: ReportPdfService computes deterministic 64-char hex SHA-256 digest ..
00:00 +21: Audit digest exhibits cryptographic avalanche effect on data change .
00:00 +22: ReportPdfService builds valid PDF document bytes with %PDF header ...
00:00 +23: WazuhRepository returns structured agent inventory in Demo Mode .....
00:00 +24: WazuhRepository returns cluster daemon statuses in Demo Mode .....
00:00 +25: WazuhRepository restartAgent and restartDaemon actions succeed .....
00:00 +26: WazuhAgentModel serialization and deserialization validation .....
00:00 +27: WazuhDaemonModel serialization and deserialization validation .....
00:00 +28: OfflineMutation model serialization and deserialization .....
00:00 +29: OfflineQueueService enqueues, queries, and removes mutations .....
00:00 +30: OfflineQueueService flushQueue handles empty queue gracefully .....
00:00 +31: OfflineMutation copyWith creates modified clones correctly .....
00:02 +32: SecurePulse Mobile app pump test .....
```

Total Results: 33 Passed, 0 Failed, 0 Skipped (100% Pass Rate)

6.3 Static Analysis & Code Quality Results

Execution of `dart analyze` across all codebase files yielded:

```
Analyzing securepulse_mobile...  
No issues found!
```

This confirms zero unused imports, complete null-safety conformance, and strict adherence to Flutter engineering standards.

CHAPTER 7 — CLOUD DEPLOYMENT & RELEASE ENGINEERING

7.1 Cloud Backend Deployment (FastAPI on Render Cloud)

The backend microservice is deployed as a containerized Python 3.11 ASGI service on **Render Cloud** (<https://secureguard-backend-7eqm.onrender.com>). It connects to a managed PostgreSQL 16 database with SSL enforcement and connection pooling. The deployment pipeline features automated Git CI/CD auto-deploying on every push to the `main` branch.

7.2 Android Production Compilation & ProGuard / R8 Optimization

The production Android release build was compiled using `flutter build apk --release`. ProGuard and R8 shrinking rules in `[android/app/proguard-rules.pro]` (`file:///e:/SOC%20projects/securepulse-mobile/android/app/proguard-rules.pro`) removed unused code while preserving required reflection classes for Flutter embedding, KeyStore cryptography, and BiometricPrompt.

Metric	Unoptimized Build	ProGuard / R8 Optimized Release	Improvement
APK Binary Size	63.6 MB	**57.1 MB**	**~10.2% Reduction**
MaterialIcons Font	1,645 KB	**12.4 KB**	**~99.2% Tree-Shaken**
CupertinoIcons Font	257 KB	**0.8 KB**	**~99.7% Tree-Shaken**
Compilation Time	140.2s	**130.7s**	**Optimized Build**

7.3 Google Play Store Packaging & Keystore Signing Configuration

The Android build configuration in [`android/app/build.gradle`] (file:///e:/SOC%20projects/securepulse-mobile/android/app/build.gradle) implements a robust `signingConfigs.release` pattern loading credentials from `key.properties`, with an automated fallback to debug signing for development/CI environments. A template is provided in [`android/key.properties.example`](file:///e:/SOC%20projects/securepulse-mobile/android/key.properties.example) for 2048-bit RSA keys generated via `keytool`.

CHAPTER 8 — LIMITATIONS, FUTURE WORK & CONCLUSION

8.1 Technical Constraints & Operational Boundaries

- **Firestore Cloud Messaging Setup:** Remote cloud push notifications require supplying a registered project `google-services.json`. In its absence, the app falls back safely to local in-memory event streams without application crash.
- **Edge Execution Delegation:** Incident containment actions (IP quarantine) are dispatched to backend event workers rather than executing direct mobile-to-firewall API calls.

8.2 Future Evolutionary Roadmap & Autonomous SOAR

- **Autonomous PR Generation:** Automatically creating GitHub Pull Requests with verified patches based on AI Copilot recommendations.
- **Multi-Cloud SOAR Connectors:** Building native connectors for AWS Network ACLs, Cloudflare WAF, and Cortex XSOAR.
- **Multi-Tenant MSSP Portal:** Seamless organization switching with tenant-scoped cryptographic keys.
- **On-Device Quantized SLMs:** Running quantized Gemma-2B / Llama-3-1B via ONNX Runtime for 100% offline conversational AI triage.

8.3 Concluding Remarks

SecurePulse successfully demonstrates that full-featured, enterprise-grade cybersecurity operations can be transitioned into a mobile-first paradigm without compromising security, speed, or cryptographic non-repudiation. By unifying Wazuh SIEM telemetry, Semgrep SAST auditing, conversational AI assistance, and SHA-256 audit stamping into an encrypted, biometrically secured Flutter client, the platform delivers a comprehensive, production-ready mobile SOC console.

REFERENCES & APPENDICES

Appendix A: REST & WebSocket API Endpoint Specifications

- `GET /health` — Service uptime probe and database health check.
- `POST /v1/auth/login` — Analyst credential exchange returning JWT bearer token.
- `POST /v1/auth/github` — OAuth2 PKCE authorization code exchange.
- `GET /v1/dashboard/summary` — Posture gauge score and severity telemetry counts.
- `GET /v1/repositories` — Monitored codebase inventory with health grades.
- `POST /v1/repositories/{id}/scan` — On-demand SAST scan trigger dispatch.
- `GET /v1/soc/alerts` — Paginated SOC intrusion alert stream.
- `PUT /v1/soc/alerts/{id}/status` — Alert lifecycle mutation (`open` → `resolved`).
- `GET /v1/wazuh/agents` — Wazuh connected agent inventory inspection.
- `POST /v1/wazuh/agents/{id}/restart` — Remote Wazuh agent restart command.
- `WS /ws/alerts` — Full-duplex WebSocket threat incident stream.

Appendix B: References & Academic Standards Bibliography

- **NIST SP 800-61 Rev. 2:** *Computer Security Incident Handling Guide*, National Institute of Standards and Technology, U.S. Department of Commerce.
- **RFC 7519:** *JSON Web Token (JWT)*, Internet Engineering Task Force (IETF), Standards Track.
- **RFC 7636:** *Proof Key for Code Exchange by OAuth Public Clients (PKCE)*, Internet Engineering Task Force (IETF).
- **OWASP Mobile Top 10 (2024):** *Mobile Application Security Verification Standard (MASVS)*, Open Web Application Security Project.
- **Wazuh Platform Architecture:** *Host-based Intrusion Detection System Specification & Ruleset v4.8*, Wazuh Documentation.

- **Semgrep OSS Documentation:** *Static Analysis Engine & Semantic Pattern Matching*, Return To Corporation.
- **Flutter Framework Specification:** *Reactive UI Architecture and State Management with Riverpod*, Google LLC.